

not created....” to indicate that the message queue creation has failed. Figure 4 shown the message creation program. The list of header files showed in the program is the same as in the remaining programs discussed in this article.

Send a message

Sending a message to a message queue involves the library function *mq_send*. This function accepts four parameters. The first parameter specifies the message queue descriptor pointing to a valid or pre-existing message queue, the second is a reference to a message pointed by a *msg_ptr*, the third describes the length of the message that is being sent to the receiving process, and the last is used to define the message that needs to be received from the queue depending on the priority of the message that has been mentioned.

The sending process blocks if the message queue has no space. It resumes when the queue contains enough memory for a new message. The messages in the message queue are stored in a way such that they are in a decreasing order of priority. If the *O_NONBLOCK* flag is specified in the *mq_send* system call, then the process terminates returning -1, setting the appropriate *errno*.

In the code snippet shown in Figure 5, we have opened a message queue that already exists — */mymq*. The sending process opens the queue in write-only mode (*O_WRONLY*) and stores the descriptor in *mq_snd_open*. We then prompt the user to enter a message he/she intends to send to the queue and store it in a buffer *buf*. We define *len* as the length of the message being sent.

In the *mq_send* library function, we pass the message queue descriptor *mq_snd_open*, the buffer *buf* which stores the message, the length of the buffer+1 (which can be avoided by mentioning the null terminator at the end of the message) and the priority of the message being sent (which is 0 here). The return value of this call is stored in *mq_ret*, which returns 0 when it is successful (here we print “Message is sent successfully...”). Else, we print “Message is not sent” indicating that the call was unsuccessful.

```
int main ( ) {
    mqd_t mq_create, mq_snd_open;
    int mq_ret;
    long int len;
    char buf[8192];

    mq_snd_open=mq_open("/mymq", O_WRONLY);

    printf ("Please enter the message : ");
    scanf ("%s", buf);
    len=strlen(buf);

    mq_ret=mq_send(mq_snd_open, buf, len+1, 0);
    if(mq_ret == 0)
        printf (" message is sent successfully...\n");
    else
        printf (" Message is not sent ....\n");
}
```

Figure 5: Sending messages in POSIX message queues

Receiving a message

The *mq_receive* library function takes four parameters, the same as *mq_send*. This function blocks by default until the message queue is populated with messages. If *NON_BLOCK* is mentioned, then the process fails by setting the *errno* to *EAGAIN*. If the call is a success, it returns with the number of bytes of the message received; else, it fails with -1 and the *errno* is set.

The receiving process requires opening the message queue by providing the name of the message queue, in this case ‘*/mymq*’, which is opened in read-only mode (*O_RDONLY*). Once the message queue has been opened and the descriptor is stored in *mq_rec_open*, we pass this to the *mq_receive* system call along with the other parameters. The message in the queue is stored in the buffer *buf* of the receiving process, which is then printed (as shown in Figure 6).

```
int main ( ) {
    mqd_t mq_rec_open;
    char buf[8192];
    int ret_rec;

    mq_rec_open=mq_open("/mymq", O_RDONLY);
    ret_rec=mq_receive(mq_rec_open, buf, sizeof(buf), NULL);
    printf ("Message: %s\n", buf);
}
```

Figure 6: Receiving messages in POSIX

Sending a message with priority

The most important advantage of using POSIX message queues is that messages can be prioritised. Processes may send or extract messages from the queue based on the priority. We shall explore how to make use of this feature in this section.

POSIX priority ranges from 0 – *sysconf(_SC_MQ_PRIO_MAX)-1*, where ‘*sysconf(_SC_MQ_PRIO_MAX)-1*’ is 32767. For applications implementing POSIX message queues, the message with the highest priority is delivered immediately to the receiving end. The system call eats up the message with the highest priority in the message queue referenced by the message queue descriptor that has been provided in the system call. The messages in the message queue are stored in such a way that they are in a decreasing order of priority.

In the code snippet given in Figure 7 of the sending process, we open the message queue that already exists in write-only mode (*O_WRONLY*). We prompt the user to send a message and a priority of the message that needs to be sent. The change made in the *mq_send* call is that the fourth parameter, indicating the priority, is now a valid integer *prio*. If the *mq_ret* call returns 0, the message has been sent successfully; else, it has failed and we print the appropriate messages.

Finally, in the receiving process we open the message queue in read-only mode (*O_RDONLY*). The only modification to the *mq_receive* system call made is the fourth parameter of the *mq_receive* of a particular priority. We then